Coding cheat sheet

Darya Yuferova

First steps

- Declare a working directory `setwd("file.path")`.
- If you use a package (e.g., `plm`), you have to install it `install.packages("plm")` and then initiate it for every session `require(plm)` or `library(plm)`.
- For different data types please see intro lecture to R.

Saving and loading objects

- If you want to save R objects, use `save(objects, file = "xxx.RData")`.
- If you have an `.RData` item, use `load()` to load it back into memory.
- If you read a `.csv` or `.txt` file, use `read.table()` or `read.csv()`. Make sure you chose the right field separation character, e.g., for tab-delimited files use `sep = "\t"`.
- If you want to write a data frame to `.csv` or `.txt`, use `write.csv()` or `write.table()`.
- If you use `.csv` or `.txt` files, the relevant parameters that you might have to set are: (1) field separator, (2) row and column names, and (3) quotes around entries.
- Potential issues if numerical values are not recognized: (1) Norwegian/local versus English convention of decimal separators, (2) empty last line, and (3) field separator used as character in a specific cell which leads to an uneven column count.

Referencing subsamples in the data

- Assume you have a data.frame `data` which has two columns `data$count` and `data$time`.
- Extract only a certain column from the data.frame: `data$count`, `data[,"count"]`, `data[, 1]`, `data %>% select(count)`, and `data %>% pull(count)` do the job.
- Extract only certain rows given a condition on `data$time`: `data[data$time == ,]`, `subset(data, time == )`, and `data %>% filter(time == )` do the job.

Winsorizing

- Winsorizing sets all values above or below a certain threshold to this threshhold. You need to use the `DescTools` package and following syntax: `Winsorize(variable, probs = c(lower.threshold, upper.threshold), na.rm = T/F)`. `na.rm` is used to deal with missing values in the data.
- You may visualize the effect by comparing `hist(variable)` and `hist(winsorized.variable)`.

OLS regression

- Assume that you have a data.frame `my.data` which has two columns `x` and `y`.
- Estimate standard OLS: `fit <- lm(y ~ x, data = my.data)`.
- Inspect output from OLS object: `summary(fit)`.
- Extract coefficients, standard errors, t-statistics, and p-values (matrix with columns with items as ordered) `summary(fit)$coefficients`. E.g., extract only standard errors `summary(fit)$coefficients[,2]`.
- Predict for new data `new.data` which has the same two columns `x` and `y`: `predict(fit, new.data)`.

Multiple hypothesis testing

- You need the package `car`.
- Multiple restrictions: Assume that you have an OLS regression `fit <- lm(y ~ x1 + x2, data = ...)` and you are interested if β_1 and β_2 are jointly zero. You need to specify your hypothesis `myH0 <- c("x1=0", "x2=0")`. Implement an F-test `linearHypothesis(fit, myH0)`.
- Single restriction on multiple regressors: assume that you have an OLS regression `fit <- lm(y ~ x1 + x2, data = ...)` and you are interested if $\beta_1 = \beta_2$. You need to specify your hypothesis `myH0 <- c("x1=x2")`. Implement an F-test `linearHypothesis(fit, myH0)`.

Panel data regressions

- You need to use the `plm` package.
- Declare a panel data.frame `my.data <- pdata.frame(my.data, index = c("entity", "time"))`. A panel has two dimensions entity and time. You need to specify which variables in `my.data` are indices for entity and time.
- Check properties of panel `pdim(mydata)`. Indicates how many entities and time periods there are. Balanced panels: all units are tracked over all time periods. Unbalanced: everything else.
- Panel data regression: `plm(y ~ x, data = , model = , effects = )`
  - `model` can take three relevant values: `pooling` OLS regression, `fd` differences the dependent variable on time periods, `within` estimates within variation by including fixed effects.
  - If `model` equals `within` you can specify which fixed effects you want: for individual entities `individual`, for time periods `time`, and for both `twoways`.
  - Nothing prevents you from using `lm()` for panel data analysis as well. You would need to include fixed effects using `+ factor(...)` in the regression formula with `...` indicating the variable you want to base your fixed effects on. Note that using `lm()` is substantially slower than `plm`.
  - Please see lecture slides for equivalence between `plm` and `lm`.

Logit and probit regression

- Estimate a logit/probit regression: `glm(y ~ x, data = , family = binomial(link = logit/probit))`. `glm` stands for generalized linear regression. `family` indicates the error distribution and the link function. We use a binomial model as we have dummy variables as y. Depending on what model you want to estimate, specify logit or probit.
- Measure of fit for a logit/probit model is the MacFadden R^2. Estimation: `PseudoR2(fit)` from `DescTools` package. `fit` holds the output of a `glm()` estimated logit/probit model.

- Estimating marginal effects for logit/probit. You need the `mfx` package. `probitmfx(y ~ x, data = , atmean = T/F)` for probit. For logit use `logitmfx(...)`. If `atmeans = T`, marginal effect for the average observation in the output. If `atmeans = F`, then average marginal effect in the output.
- Multiple hypothesis test: You need the `lmtest` package. Assume that you have an OLS regression `fit <- glm(y ~ x1 + x2 + x3 +,...)` and you are interested if β_1 and β_2 are jointly zero. Estimate the restricted model `fit.restr <- glm(y ~ x3 +,...)` excluding the coefficients you want to test. Use likelihood ratio test: `lrtest(fit, fit.restr)`.

Instrumental variable regression

- Assume that you have a data.frame `my.data` with dependent variable `y`, an endogeneous regressor `x`, an instrument `z`, and additional controls `z`. Estimation:
  - `AER` package: `ivreg(y ~ x + w | z + w, data = my.data)`
  - `plm` package: `plm(y ~ x + w | z + w, data = my.data, model = ...)`
- For both functions the logic is to omit the instrument before the pipe `|` and include it after the pipe. The endogenous regressor is included before the pipe, but omitted after it.
- For the F-test see multiple hypothesis testing.

Robust standard errors

- You need the `car`, `lmtest`, and `sandwich` package.
- Assume that you have an OLS regression `fit <- lm(y ~ x1 + x2, data = ...)`.
- You want to compute heteroskedasticity robust standard errors, use: `coeftest(fit, vcov = vcovHC(fit))[,2]` or `coeftest(fit, vcov = hccm)[,2]`. `[,2]` refers to the second column of the matrix of the output which holds the standard errors.
- You want to compute autocorrelation and heteroskedasticity robust standard errors, use: `coeftest(fit, vcov = vcovHAC(fit))[,2]`. `[,2]` refers to the second column of the matrix of the output which holds the standard errors.
- You want to compute clustered standard errors in a panel set-up: `coeftest(fit, vcov = vcovHC(fit, cluster =... ))`. Cluster can be either `"group"` or `"time"` for clustering on entities or time. You need the `plm` package.
- Implement an F-test robust to heteroskedasticity `linearHypothesis(fit, myH0, vcov = vcovHC(fit))`.
- Implement an F-test with clustered standard errors `linearHypothesis(fit, myH0, vcov = vcovHC(fit, cluster = ...))`.

Constructing matched control samples

- You need the `MatchIt` package
- Let's assume you have a data.frame `my.data` with the columns `treat` indicating treatment and `x1` & `x2` being two covariats that in your opinion describe whether or not a firm was treated. In a first step, you use a logit explaining treatment. In a second step, you use predicted treatment to select firms that where not treated but given your logit regression were very likely to be treated.
- You can run this in one go, using `matchit(treat ~ x1 + x2, data = , method = "nearest", distance = "logit", ratio = k)`. `k` indicates how many controls firms are selected for each treated firm. Please refer to the documentation for more description on which methods and distances are available.

- Let's say `fit.m` holds the output of `matchit()`. You can find the matched firms using `matching$match.matrix`. Each entry is the row name of the selected control firm for each treated one.

Reporting OLS output

- Use the `stargazer` package.
- Assume you have a regression output `fit`.
- General syntax: `stargazer(fit, type= , keep.stat=c(), report=())`.
- `type`: indicates which output type: `text`, `html`, and `latex` (If you do not use latex, you might want to learn that eventually).
- `keep.stat`: specifies which model statistics shall be reported.
- `report`: determines whether and in which order variable names `v`, coefficients `c`, standard errors `s`, test statistics `t`, significance stars `*`, and p-value `p` are reported. `'vc*t'` for example specifies following order: variable name, coefficients, significance, and t-value.
- Use documentation for information on all other options: `stargazer`.
- In some cases, you might be interested in specifying new standard errors (e.g., robust standard errors). Assume `se.fit` includes the standard errors of the regressions output `fit`. Right syntax to replace standard errors in the output: `stargazer(list(fit), se = list(se.fit), ...)`.